

Feedback Loops in the Agent Bestiary

Date: 2026-05-15

Status: Reference — describes the five adaptive feedback loops as actually implemented.

Framing

A feedback loop is *negative* in the control-systems sense when the output signal is fed back to reduce the error between current behaviour and desired behaviour. Every loop described here does this: it measures a deviation from some target (coherence, accuracy, persona fidelity, team composition, routing quality), and the loop corrects toward it.

The word "negative" here does not mean harmful — it means stabilising and self-correcting. A thermostat is a negative feedback loop. So is evolution. So, if these loops work as designed, is a well-run Agent Bestiary composition.

What makes these loops *adaptive* rather than merely reactive is that the correction changes the internal state of the agent or composition permanently, not just its behaviour on the next turn. The agent that dreamed last night reasons differently today. That is adaptation.

The five loops

Loop 1 — Individual agent learning

Target: the agent should reason correctly about its domain, using what it has learned from past executions.

Signal: eval dimension scores (relevance, accuracy, completeness, persona_fidelity, forecast_calibration, etc.) written to `eval_signals` per evaluator per episode.

Correction path:

```
Agent executes → episode stored → EvaluatorRegistry scores it
  → eval_signals, agent_timeline_entries written (inline, hot path)
  → ObservabilityWorker (background): PersonaDriftMonitor, AnomalyDetector
  → ConsolidationWorker (on-demand): DBSCAN cluster → semantic rules
                                     → ontology snapshot → dream_synopsis
  → KG context injected into next execution (kg_context.rs, every call)
```

What changes: the agent's semantic memory — the rules, entities, and facts its system prompt is enriched with before each execution. The agent that has run 50 times on market analysis questions has accumulated domain-specific rules that make its 51st response qualitatively different from its first.

Timescale: dreaming cycles. Typically hours to days depending on budget allocation.

Status: fully closed and running.

Loop 2 — Behavioral correction via HITL

Target: the agent's behaviour should align with human judgment, especially on high-stakes or anomalous cases.

Signal: human reviewer decisions — Approve, Relabel, Intervene — applied to anomaly events surfaced by Loop 1.

Correction path:

```
Anomaly detected (drift, conflict, rupture, safety) → anomaly_events
  → HITL review queue (/observatory/hitl)
  → Reviewer acts: Intervene
  → InterventionEncoder: validate, stamp authority_weight=1.0
  → CoherenceGate: check  $\Gamma(C)$  – does this correction cohere with
                    the agent's existing belief system?
  → TwoWriteMemory:
      Write 1: synthetic episode (SyntheticCorrection, authority=1.0)
      Write 2: episode_corrections (audit trail, coherence_check,
minimum_update_set)
      AgentWide: bump_persona_version()
  → Synthetic episode enters Loop 1 → consolidated at HumanAuthority weight
  → New persona_version creates new drift baseline
```

What changes: the agent's persona — its effective belief system as encoded across its episodic memory. An agent-wide intervention marks a version boundary: before and after the correction, the agent's behaviour is measurably different (drift monitor will detect this). The correction is preserved in the immutable audit trail.

Safeguards: the coherence gate blocks corrections that would create incoherence in the agent's world model ($\Gamma(C) < 0.5$). Agent-wide corrections require a second independent reviewer.

Timescale: human-initiated, but the effect propagates in the next dreaming cycle.

Status: fully closed and running.

Loop 3 — Workspace coherence

Target: a workspace's multi-agent conversation should produce coherent, evidence-grounded outputs without suppressing productive disagreement.

Signal: $\Gamma(C)$ — the global coherence score from TEC settling — plus per-principle scores that distinguish productive incoherence (low P6 with high P4) from destructive incoherence (low P2, low P7).

Correction path (inner — per session):

```
Workspace messages accumulate
  → Auto-coherence evaluation every N messages (COHERENCE_AUTO_EVAL_INTERVAL)
  OR user triggers via Coherence shelf
  → TEC settling engine →  $\Gamma(C)$  + principle scores
  → cohere_and_coordinate invoked (Recommendations or Dream Notes tier):
    Stage 0: reads/updates intention map, feeds IntentionAligns/Conflicts
    Stages 1-3: diagnose → coordination brief → written to workspace
  → Agents read the coordination brief in their next turn context
```

Correction path (outer — across sessions):

```
cohere_and_coordinate accumulates session episodes in its own memory
  → Composition Dreaming (manual trigger): Stage 4 — Tension Audit
  → Valence homophily check: spread < 0.25 on arousal or valence axis?
  → Chronic destructive incoherence pattern detected?
  → propose_composition_change → composition_versions (pending)
  → Owner accepts/rejects (rejection stored as correction episode → Loop 1)
  → Accepted: teams.member_agent_ids updated, new team is live
```

What changes:

- Inner loop: the direction of the next few turns — agents receive a coordination brief that redirects them.
- Outer loop: the composition itself — team membership, member weights.

Timescale: inner loop runs within the session (minutes). Outer loop requires accumulated session history and human approval (days to weeks).

Status: inner loop fully closed. Outer loop structurally closed; operationally nascent — needs session history to accumulate before Stage 4 produces meaningful proposals.

Loop 4 — Composition evolution (tune-team RSI)

Target: the composition's team structure should improve over time to reduce chronic coordination failures and valence homophily.

Signal: recurring patterns in cohere_and_coordinate's consolidated memory — which TEC principles are chronically weak, whether the team's valence distribution has collapsed, whether destructive incoherence is persistent.

Correction path:

```
cohere_and_coordinate sessions → episodes in strategist's memory
  → ConsolidationWorker: clusters session-observation episodes
                        → team-effectiveness rules in ontology
  → Composition Dreaming (POST /api/workspaces/:id/composition/dream):
    Read consolidated memory
    Compute valence spread: arousal max-min, valence max-min
    Classify chronic incoherence type
    If structural issue: call propose_composition_change
  → composition_versions row (status=pending)
  → Owner: Accept → teams updated | Reject + note → episode in strategist
memory
  → Rejection feeds back into Loop 1 for the strategist
```

What changes: the team structure — who is in the composition, and with what weight.

Timescale: weeks to months. The strategist needs enough session history to distinguish persistent patterns from noise before making meaningful proposals.

Status: structurally closed. Data accumulation required before it becomes meaningful.

Important distinction from Loop 3: Loop 3's inner iteration changes conversation direction (fast, within-session). Loop 4 changes team composition (slow, across-sessions). They operate at different timescales and different levels of the system.

Loop 5 — Brier calibration and routing accuracy

Target: the platform's forecasting agents should become more calibrated over time; the MoE routing strategist should learn which members are genuinely accurate on which sub-domains.

Signal:

- For forecasting agents: Brier score when `fermi_forecasts` resolve against actual outcomes. Computed by `BrierEvaluator` and written to `eval_signals.dimension = "forecast_calibration"`.
- For routing decisions: the accuracy of routing decisions recorded by `moe_router_strategist` — did the chosen member produce the best outcome for this query type?

Current state of the signal path:

```
Agent executes forecast question
  → BrierEvaluator reads fermi_forecasts filtered on agents_used
  → Computes 1 - brier_score → forecast_calibration dimension
  → Written to eval_signals
  → Appears in agent_timeline_entries.dim_scores
  → Observable in observatory trend charts

moe_router_strategist routes query to member
  → Records routing decision as episode: {query_type, member_selected,
rationale, confidence}
  → ... outcome arrives (resolved forecast, SOSA observation, HITL
correction) ...
  → BREAK: no mechanism yet reads outcome and updates routing classifier
```

What is missing: The signal is collected and stored. The loop is not closed. Closing it requires connecting the outcome signal (Brier score, SOSA observation, HITL correction) back to the routing weights for future decisions. See §3 for the strategy.

Timescale: months. Requires sufficient resolved forecasts to establish calibration curves.

Status: signal collected, feedback path not wired.

The hierarchy

The five loops operate at different timescales and different system levels:

Timescale	Loop	Level	Status
Hours	1. Individual learning	Single agent	✓ Closed
Days	2. HITL correction	Single agent	✓ Closed
Session	3. Coherence (inner)	Composition chat	✓ Closed
Weeks	3. Coherence (outer)	Composition team	✗ Nascent
Months	4. Composition evolution	Team structure	🔧 Structural
Months+	5. Brier calibration	Platform-wide	🔒 Signal only

They are nested: Loop 2 feeds into Loop 1 (corrections become episodes). Loop 3's outer iteration feeds into Loop 4 (session patterns drive composition proposals). Loop 5, when closed, will feed into Loop 3 (calibration scores will weight the MoE routing classifier, which will influence which members are recommended in composition proposals).

3. Strategy for closing Loop 5

The data problem

You are right that there is a data problem. The Brier signal is thin right now because:

1. **Few resolved forecasts** — 8 agents have `fermi_contract`, meaning they produce CEP-structured output that can be scored against Polymarket or other resolution sources. But resolution takes time (forecasts on events that haven't happened yet), and the current forecast volume is low.
2. **Routing decisions are not yet annotated with outcomes** — `moe_router_strategist` records routing decisions as episodes, but there is no path yet from "this query was routed to member X" → "member X's output was evaluated as Y" → "update routing classifier."
3. **The calibration curve needs volume before it's meaningful** — a single agent's Brier score on 5 resolved forecasts is not a calibration curve. The `BrierEvaluator` already saturates confidence at n=20 resolved forecasts. Below that, the signal is informative but not actionable.

The strategy — four steps, in order

Step 1 — Bootstrap synthetic calibration data (now, without waiting)

For the forecasting agents that already have `fermi_contract`, run historical backtests: replay past Polymarket events through the agent and score the outputs. These are "synthetic" in the sense that the questions are known-resolved, but the scoring is real. This gives each forecasting agent a starting calibration curve within days, not months.

The infrastructure exists: `BrierLookupSqlx` reads from `fermi_forecasts` filtered on `agents_used`. The question is just seeding the table with historical backtest results. This is a one-time data migration, not a code change.

Step 2 — Route calibration scores into the MoE classifier (the feedback path)

Add a `GET /api/agents/:id/calibration` endpoint that returns:

```
{
  "agent_id": "...",
  "forecast_calibration_mean": 0.73,
  "forecast_calibration_trend": "improving",
  "domain_scores": {
    "market_analysis": 0.78,
    "regulatory": 0.61,
    "macroeconomic": 0.70
  },
  "n_resolved": 23
}
```

The `moe_router_strategist` 's Stage 0 classification criteria already lists "historical accuracy" as the third priority. Right now it can only read from episodic memory via `search_knowledge` . Adding a `get_agent_calibration` tool to its `mcp_tools` means the routing classifier can explicitly weight members by their measured calibration on domain-matched queries. No ML required — it's a lookup.

Step 3 — Annotate routing decisions with outcomes

When a routed query resolves (forecast resolves, SOSA observation arrives, HITL correction applied), write back to the routing-decision episode: "this routing decision produced outcome quality Y." The `moe_router_strategist` 's `search_knowledge` call in Stage 0 can then find past routing decisions for similar queries and weight toward members that resolved well.

This is the pure episodic memory path — no separate routing table needed. The episode context already holds `{query_type, member_selected, confidence}` ; adding `{outcome_quality, outcome_source}` when the resolution arrives closes the loop through the existing ADM pipeline.

Step 4 — Let Loop 1 do the rest

Once routing decisions are annotated with outcomes, Loop 1 (individual agent learning) handles the rest. The `moe_router_strategist` 's own dreaming cycle consolidates routing-decision episodes into rules like "for macroeconomic questions, macro_forecaster has historically outperformed sentiment_analyzer by 0.12 Brier points." These rules enter its KG context and are read in future Stage 0 classifications.

This is the key insight: **Loop 5 closes through Loop 1**. The MoE strategist is itself an agent that learns via dreaming. The calibration signal is just a new dimension of evidence that its episodic memory can consolidate. The only engineering work is: (a) the calibration endpoint, (b) the outcome annotation on routing episodes, and (c) the `get_agent_calibration` tool on the strategist's card.

On the cold-start problem

The data problem is real, but it's bounded. The platform does not need calibration data to be useful — it needs calibration data to be *learning*. A composition without Loop 5 closed still works; it just routes based on declared capabilities rather than demonstrated accuracy. That is a reasonable starting state.

The progression is:

- **Month 0–2:** routing based on `accepts` / `produces` / `skills` declarations (semantic matching, deterministic)
- **Month 2–4:** routing increasingly weighted by historical accuracy as forecasts resolve and backtest data is seeded
- **Month 4+:** routing is a calibrated probabilistic classifier, and composition proposals from Loop 4 are informed by which members are actually accurate in which sub-domains

The value of the system increases monotonically with data. The architecture does not break at low data volume — it degrades gracefully to semantic matching. That is the right design.

4. What makes this architecture coherent

Each loop corrects at the appropriate timescale:

- Fast loops (1, 2, inner-3) handle execution-level errors — the agent said the wrong thing, the team went in the wrong direction.
- Slow loops (outer-3, 4) handle structural errors — the team is wrong for the problem, the composition needs to change.
- Calibration loop (5) handles systematic bias — the routing classifier has persistent blind spots that need data to reveal.

Each loop uses a different corrective mechanism:

- Loops 1 and 2: episodic memory → dreaming → semantic rules
- Loop 3: TEC coherence → coordination brief → conversation steering
- Loop 4: session patterns → composition proposals → human approval → team change
- Loop 5: calibration scores → routing weights → member selection

Each loop is separated from the others by a human or coherence gate:

- Loop 2 requires a human reviewer (anomaly → HITL queue)
- Loop 4 requires owner approval (composition proposal → accept/reject)
- Loop 5's routing weights are readable by humans via the calibration endpoint

No loop can modify agent behaviour without either a human gate or the coherence gate. This is not an accident. It is the property that makes the system trustworthy as it learns.